Universidade Federal Rural de Pernambuco – UABJ

Engenharia da Computação

RELATÓRIO DO PROJETO

Modelagem e Programação Orientada a Objetos

Identificação do projeto

Campo	Preencher
Título do projeto	Projeto 2: Urna Eletrônica
Semestre	2026.2
Professor	Michael Oliveira da Cruz
Repositório Git (link)	https://github.com/helenamartins252/Projeto-de-Programacao-2---Urna-Eletronica.git

Integrantes da equipe

*Preencha o nome completo de cada integrante. *Linha 4 apenas se a equipe tiver quatro integrantes.*

Nº	Nome completo	Matrícula	E-mail
1	Julia Gabriela de Andrade Barboza	20250029167	julia.barboza@ufrpe.br
2	Raiane Mayra Freitas Cintra	20250031835	raiane.cintra@ufrpe.br
3	Maria Luisa Marcelina de Meirelles	20250029238	luisameirelles7@gmail.com
4*	Maria Helena da Silva Martins	20250023619	mariahelenahellena124@gmail.com

Controle de entregas

Preencha a data e a tag do Git ao concluir cada entrega. A coluna de nota é do professor.

Entrega	Data de entrega	Tag no Git	Nota (professor)
1. Modelagem	12/11/2026	entrega1	
2. Núcleo do domínio	13/11/2026	entrega2	
3. Persistência e funcionalidades		entrega3	
4. Versão final		entrega4	

Apresentação do projeto

Preencher na Entrega 1 e manter atualizado ao longo do projeto.

Descrição do problema

Em até um parágrafo, explique o que o sistema faz e para quem.

O sistema simula uma eleição por meio de uma urna eletrônica. Os eleitores votam em candidatos usando a urna, digitando o número do candidato e confirmando, ou votando em branco. Cada eleitor pode votar apenas uma vez e o voto é secreto: depois de votar, ninguém consegue saber em quem a pessoa votou. Quando a votação termina, a urna é encerrada, conta os votos de cada candidato, os brancos e os nulos, e gera um boletim de urna em arquivo TXT. O sistema é voltado a quem organiza e simula a eleição (mesário ou administrador) e aos eleitores cadastrados.

Tecnologias utilizadas

Item	Utilizado? (sim/não) e detalhes
Interface gráfica (Swing) ou menu de console	Sim, os dois, em etapas diferentes. Console na Entrega 2, só para demonstrar o domínio funcionando (votar, votar em branco, encerrar, contabilizar e tratar uma exceção). Swing na Entrega 4, com as três telas: Identificação (título ou CPF), Votação (número, candidato na tela e botões Confirma, Corrige e Branco) e Apuração (votos, brancos, nulos, total e botão de gerar o boletim). Erros aparecem em caixa de mensagem (JOptionPane) e o programa continua.
Banco de dados (SQLite / JDBC)	Sim, a partir da Entrega 3. SQLite acessado por JDBC, num único arquivo de banco. Tabelas eleitor, candidato, urna, urna_candidato e voto, criadas sozinhas na primeira execução. Uma classe DAO para cada tabela, que captura a SQLException e lança uma exceção do próprio sistema, assim a tela não sabe nada de SQL. Marcar ja_votou e gravar o voto são feitos numa só transação JDBC (commit se tudo der certo, rollback se algo falhar), para cumprir a RN.5. A tabela voto não tem coluna ligada ao eleitor (RN.2).
Arquivos (CSV, JSON, serialização, TXT)	Sim, só TXT. O boletim de urna é gerado em arquivo TXT pelo botão GERAR BOLETIM, disponível só com a urna encerrada, com a urna, a data, o total de cada candidato, os brancos, os nulos e o total geral. Não usamos CSV, JSON nem serialização.
Versão do Java e ferramentas debuild	Sim. Java 17 (versão LTS e a mínima exigida no enunciado). Build com Maven, que baixa o driver do SQLite (org.xerial:sqlite-jdbc) e padroniza a estrutura de pastas do projeto. IDE: VS Code. Testes: JUnit 5, para conferir as regras do modelo (voto único por eleitor, urna encerrada recusando voto, voto sem ligação com o eleitor). Apoio de IA: Claude (Anthropic), usado para tirar dúvidas, revisar a modelagem e sugerir trechos de código, sempre lidos, testados e ajustados pela equipe.

Requisitos funcionais

ID	Requisito	Prioridade (alta/média/baixa)
RF.1	Cadastrar eleitores e candidatos	Alta
RF.2	Cadastrar urnas e dizer quais candidatos concorrem em cada uma	Alta
RF.3	Votar digitando o número do candidato e confirmando	Alta
RF.4	Aceitar voto em branco; número que não existe vira voto nulo, com aviso	Alta
RF.5	Encerrar a urna (comando do mesário) e contar os votos, os brancos e os nulos	Alta
RF.6	Gerar o boletim de urna em TXT	Média
RF.7	Mostrar porcentagem de cada candidato (sobre o total de votos, incluindo brancos e nulos) e o vencedor	Baixa

Acrescente linhas conforme necessário.

Regras de negócio que o sistema deve cumprir:

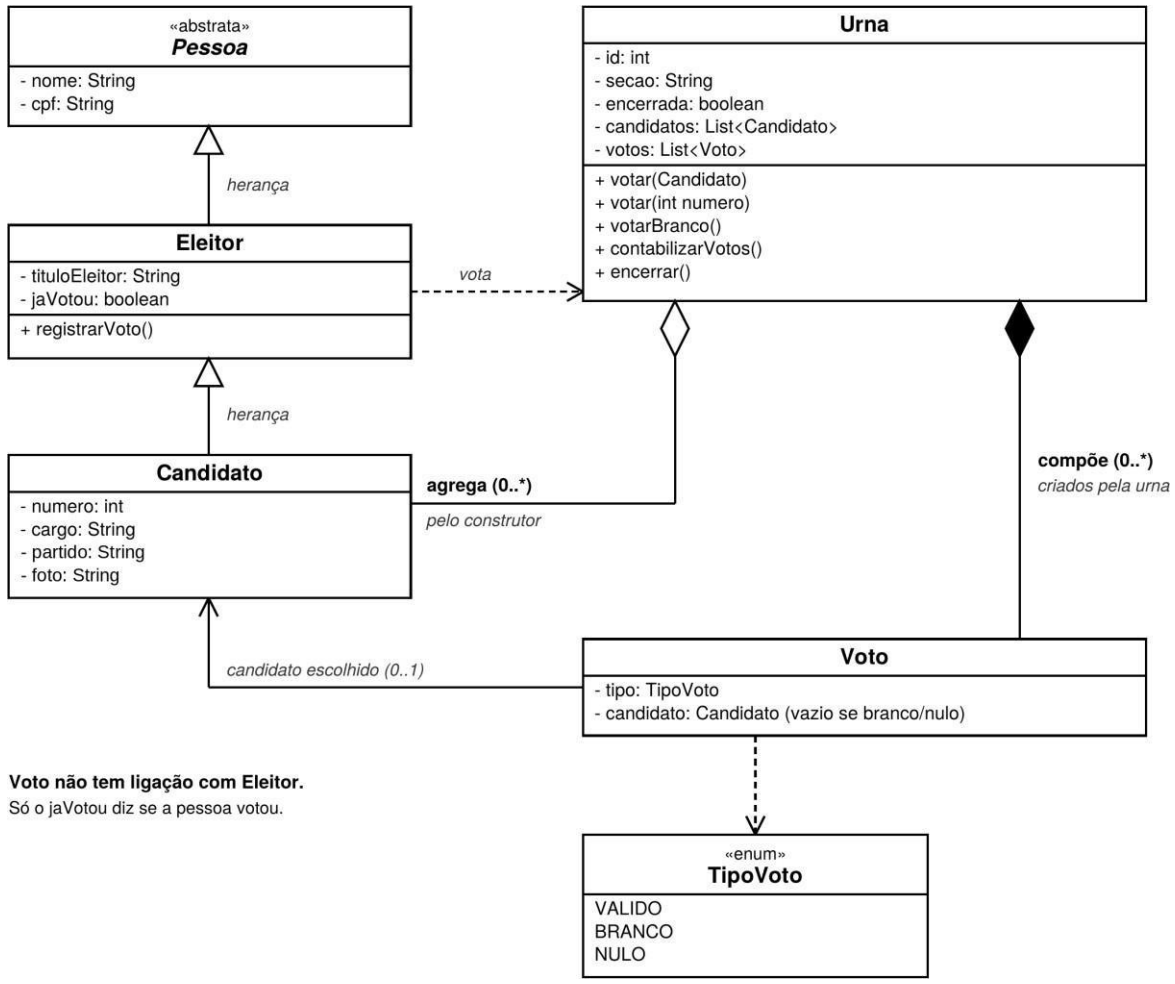
ID	Regra
RN.1	Cada eleitor vota no máximo uma vez; quem controla é o jaVotou do eleitor
RN.2	Voto secreto: nenhuma ligação entre voto e eleitor
RN.3	Urna encerrada não aceita voto
RN.4	O voto é criado dentro da urna, por votar()
RN.5	Marcar jaVotou e gravar o voto acontecem juntos
RN.6	Só eleitor cadastrado vota; só candidato ligado à urna aparece nela

Entrega 1: Modelagem

Objetivo: mostrar que o problema foi compreendido e projetado antes de programar.

1.1 Diagrama de classes UML

Cole a imagem do diagrama. Inclua atributos, métodos principais, visibilidade e multiplicidades.



1.2 Justificativa dos relacionamentos

Para cada relacionamento do diagrama, diga o tipo (associação, agregação, composição, herança) e por que ele foi escolhido. Deve haver ao menos um de cada tipo.

Classes envolvidas	Tipo	Justificativa
Pessoa e Eleitor	Herança	Todo eleitor é uma pessoa. Eleitor reaproveita nome e CPF de Pessoa e acrescenta título de eleitor e jaVotou.
Eleitor e Candidato	Herança	Todo candidato também é eleitor. Por isso a herança tem três níveis: Pessoa, Eleitor e Candidato, com super(...) preenchendo os dados da classe de cima.

Urna e Candidato	Agregação (0..*)	O candidato existe sem a urna e pode concorrer em várias urnas. A lista chega pronta pelo construtor da urna. Se a urna for apagada, os candidatos continuam existindo.
Urna e Voto	Composição (0..*)	O voto só existe dentro da urna. A lista de votos é criada na própria urna e quem cria cada Voto é a urna, nos métodos votar(). Se a urna for apagada, os votos vão junto.
Voto e Candidato	Associação (0..1)	O voto válido aponta para o candidato escolhido. Em voto branco ou nulo, não aponta para nenhum.
Voto e TipoVoto	Dependência (uso do enum)	O tipo do voto (VALIDO, BRANCO ou NULO) é um conjunto fixo de valores, por isso é um enum.
Eleitor e Urna	Associação de uso (dependência, tracejado)	O eleitor vota na urna, mas a urna não guarda o eleitor. O Voto não tem ligação com Eleitor, para o voto ser secreto (RN.2). O controle de quem já votou fica só no jaVotou, que muda por um método do próprio eleitor.

1.3 Modelo de dados ou formato dos arquivos

Banco: diagrama entidade-relacionamento ou tabelas com colunas e chaves. Arquivos: layout dos campos (ex.: cabeçalho do CSV, estrutura do JSON).

Tabela	Colunas
eleitor	id (PK), nome, cpf, titulo, ja_votou
candidato	id (PK), eleitor_id (FK), numero, cargo, partido, foto
urna	id (PK), secao, encerrada
urna_candidato	urna_id (FK), candidato_id (FK)
voto	id (PK), urna_id (FK), tipo (VALIDO, BRANCO ou NULO), candidato_id (FK, vazio se for branco ou nulo)

Observações:

- A tabela voto não tem nenhuma coluna ligada ao eleitor, então não dá para saber em quem cada pessoa votou (RN.2).
- A tabela urna_candidato liga as urnas com os candidatos (agregação).
- O candidato é um eleitor, por isso a tabela candidato aponta para eleitor.
- Marcar ja_votou e gravar o voto acontecem na mesma transação (RN.5).

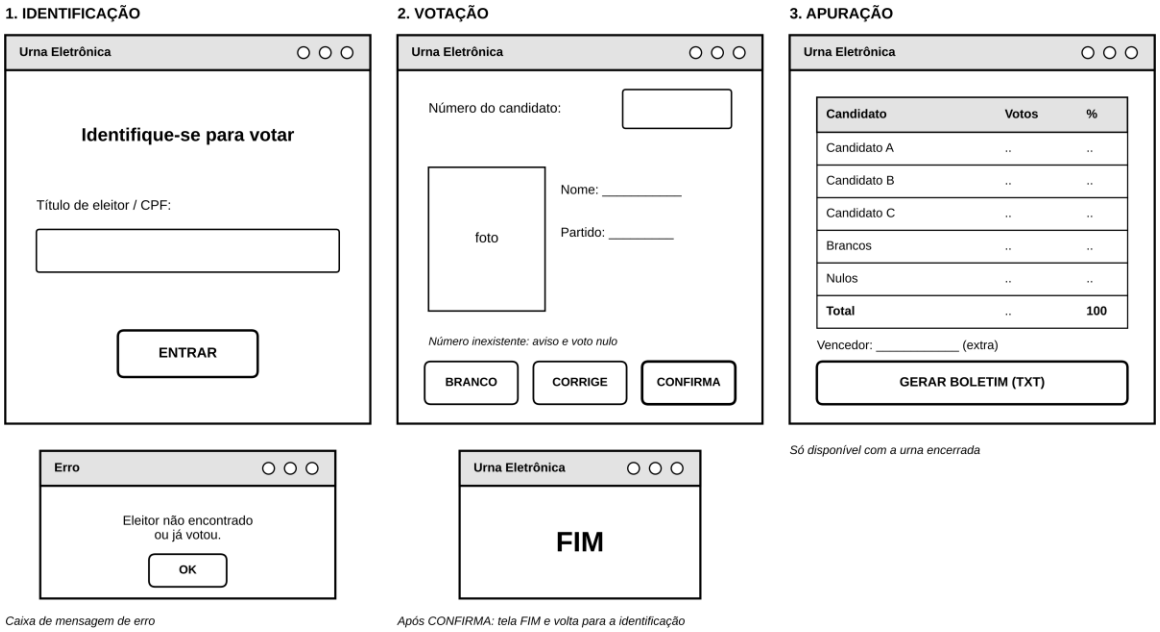
Arquivo TXT (boletim de urna): contém a urna, a data, o total de cada candidato, os brancos, os nulos e o total geral.

Exceções previstas:

Exceção	Quando é lançada
EleitorJaVotouException	O eleitor tentou votar de novo
UrnaEncerradaException	Tentou votar numa urna encerrada
EleitorNaoEncontradoException	Título ou CPF não cadastrado
CandidatoInvalidoException	O candidato não pertence à urna

1.4 Esboço das telas

Desenhos simples (papel, Figma, Swing Designer) das telas principais. Para projetos com menu de console, descreva o menu.



Descrição das três telas principais:

1. **Identificação.** O eleitor digita o título de eleitor ou CPF e clica em ENTRAR. Se não estiver cadastrado ou já tiver votado, aparece uma caixa de mensagem com o erro ("Eleitor não encontrado ou já votou") e um botão OK.
2. **Votação.** O eleitor digita o número do candidato e a tela mostra a foto, o nome e o partido. O botão CONFIRMA registra o voto, CORRIGE limpa o número e BRANCO vota em branco. Se o número não existe, o sistema avisa e o voto conta como nulo. Após CONFIRMA, aparece a tela "FIM" e o sistema volta para a identificação.
3. **Apuração.** Aparece quando a urna é encerrada e mostra uma tabela com candidato, votos e porcentagem, além de brancos, nulos e total. Também mostra o vencedor (extra) e tem o botão GERAR BOLETIM (TXT), disponível só com a urna encerrada.

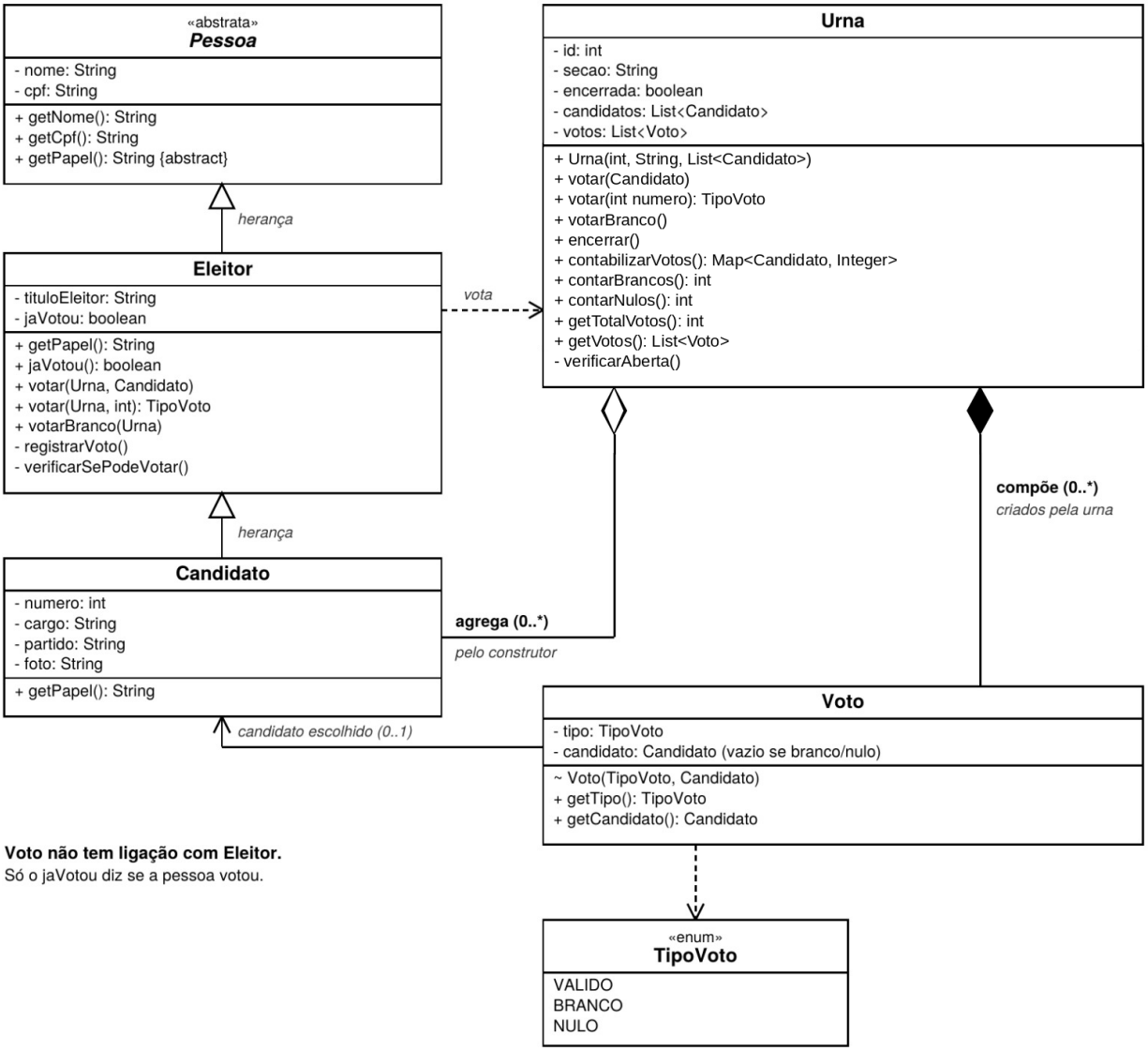
Resumo: as telas essenciais são três (Identificação, Votação e Apuração), as mesmas que o enunciado pede. A tela "FIM" que aparece depois do CONFIRMA e a caixa de mensagem de erro não são telas separadas: a primeira é só uma mensagem de encerramento do voto e a segunda é um diálogo que aparece sobre a tela atual.

Entrega 2: Núcleo do domínio

Objetivo: classes do modelo implementadas e demonstradas por um Main de console.

2.1 UML atualizado

Se o diagrama mudou desde a Entrega 1, cole a nova versão e liste abaixo o que mudou e por quê.



O que mudou em relação à Entrega 1 e porquê:

O diagrama foi atualizado em relação à Entrega 1. A estrutura (classes, atributos e relacionamentos) foi mantida; as mudanças foram nos métodos, detalhados agora que o código foi implementado.

Não mudaram: as heranças (Pessoa, Eleitor e Candidato), a agregação da urna com os candidatos, a composição da urna com os votos, o enum TipoVoto e a ausência de ligação entre Voto e Eleitor (RN.2).

Classe	O que mudou	Por quê

Pessoa	Ganhou o método abstrato getPapel() e os métodos getNome() e getCpf().	Para a classe abstrata ter um método abstrato e permitir polimorfismo entre Eleitor e Candidato. Os getters dão acesso de

		leitura aos atributos privados.
Eleitor	registrarVoto() virou privado. Entraram votar(Urna, Candidato), votar(Urna, int): TipoVoto, votarBranco(Urna), jaVotou(), getPapel() e verificarSePodeVotar().	O eleitor confere o jaVotou antes de pedir o voto à urna e só o marca depois que a urna aceitou. Assim o voto e o jaVotou andam juntos (RN.1 e RN.5) e o jaVotou só muda por método do próprio eleitor. Nesta entrega o voto e o jaVotou andam juntos em memória; a atomicidade real (transação JDBC) vem na Entrega 3.
Candidato	Sobrescreve getPapel(). Mantém numero, cargo, partido e foto.	Polimorfismo: cada classe filha diz o seu papel.
Urna	votar(int) devolve o TipoVoto. contabilizarVotos() devolve um Map<Candidato, Integer>. Entraram contarBrancos(), contarNulos(), getTotalVotos() e getVotos() (lista não modificável). Entraram o construtor Urna(int, String, List<Candidato>) e verificarAberta() (privado).	A tela precisa saber quando o voto virou nulo para avisar o eleitor. A apuração precisa de brancos, nulos e total. A lista de votos não pode ser alterada de fora. O construtor recebe a lista de candidatos (agregação). verificarAberta() concentra a checagem da RN.3 e lança UrnaEncerradaException
Voto	O construtor deixou de ser público (visível só no pacote modelo). Entraram getTipo() e getCandidato().	Só a Urna cria votos (RN.4).
Exceções	As quatro exceções viraram classes que estendem Exception, no pacote modelo.	O compilador obriga a tratar cada uma.

2.2 Conceitos de OO aplicados

Indique onde cada conceito aparece no código (classe e método) e explique em uma frase.

Conceito	Onde aparece (classe / método)	Explicação
Classe abstrata e método abstrato	`Pessoa` / `getPapel()`	**Pessoa é abstrata e declara `getPapel()` sem corpo. Não dá para criar uma Pessoa direto: ela sempre é eleitor ou candidato.**
Herança	`Eleitor extends Pessoa` e `Candidato extends Eleitor` (construtores com `super(...)`)	**Herança em três níveis: o candidato reaproveita tudo do eleitor, que reaproveita nome e CPF da pessoa.**
Polimorfismo	`Eleitor.getPapel()` e `Candidato.getPapel()` (`@Override`); a classe `Demo` percorre uma `List<Eleitor>`	**O mesmo `getPapel()` responde "Eleitor" ou "Candidato" conforme o objeto, sem usar `if` para descobrir o tipo.**
Sobrecarga de método ou construtor	`Urna.votar(Candidato)` e `Urna.votar(int)`; `Eleitor.votar(Urna, Candidato)` e `Eleitor.votar(Urna, int)`	**Mesmo nome para votar direto no candidato ou pelo número digitado.**

Encapsulamento	`Eleitor.jaVotou` e `registrarVoto()`; `Urna.getVotos()`; construtor de `Voto`	**O jaVotou é privado e só muda por método do eleitor. A lista de votos sai como lista não modificável. Só a urna cria votos.**
Princípio de Liskov	`Main` e Demo: `eleitores.addAll(candidatos)`	**Um Candidato entra onde se espera um Eleitor e `votar(...)` funciona igual, sem quebrar nada.**

Agregação ou composição	Urna (construtor e lista votos)	**Agregação: a lista de candidatos chega pelo construtor e os candidatos existem sem a urna. Composição: a lista de votos nasce dentro da urna e só ela cria os votos.**
Enum	TipoVoto / Voto.getTipo() e Urna.votar()	**Conjunto fixo de valores (VALIDO, BRANCO e NULO), evita erro de digitação.**

2.3 Exceções customizadas

Exceção	Quando é lançada	Onde é tratada
EleitorJaVotouException	`Eleitor.verificarSePodeVotar()`, quando o jaVotou já é verdadeiro (em `votar` e `votarBranco`)	`Main.votar()` e Demo, no bloco `catch`: mostram a mensagem e o programa continua

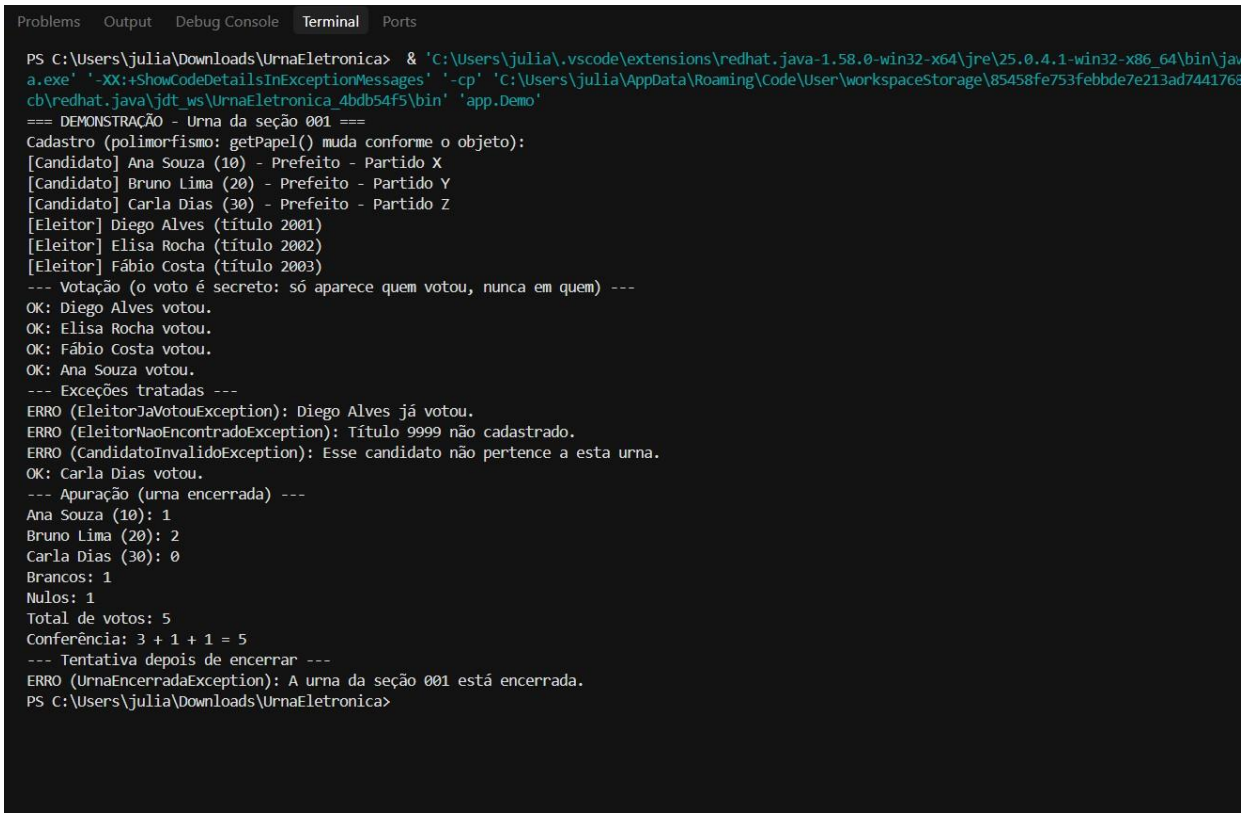
Exceção	Quando é lançada	Onde é tratada
UrnaEncerradaException	`Urna.verificarAberta()`, ao votar numa urna encerrada	`Main.votar()` e Demo
`EleitorNaoEncontradoException`	`Main.buscarEleitor()`, quando o título não está cadastrado	`Main.votar()` e Demo
`CandidatoInvalidoException`	`Urna.votar(Candidato)`, quando o candidato não pertence à urna	Demo

Na Entrega 3, a verificação de eleitor cadastrado (EleitorNaoEncontradoException) passa da classe Main para a camada dao.

2.4 Evidências de execução

Cole a saída do Main de demonstração (texto ou captura de tela), mostrando o domínio funcionando e pelo menos uma exceção tratada.

Saída do app.Demo, uma demonstração automática que não pede nada no teclado. Ela mostra cadastro em listas, voto válido, branco e nulo, as quatro exceções tratadas, o encerramento e a apuração. Como o voto é secreto, a saída mostra só quem votou, nunca em quem: votos em candidato, em branco e nulos aparecem apenas nos totais da apuração.

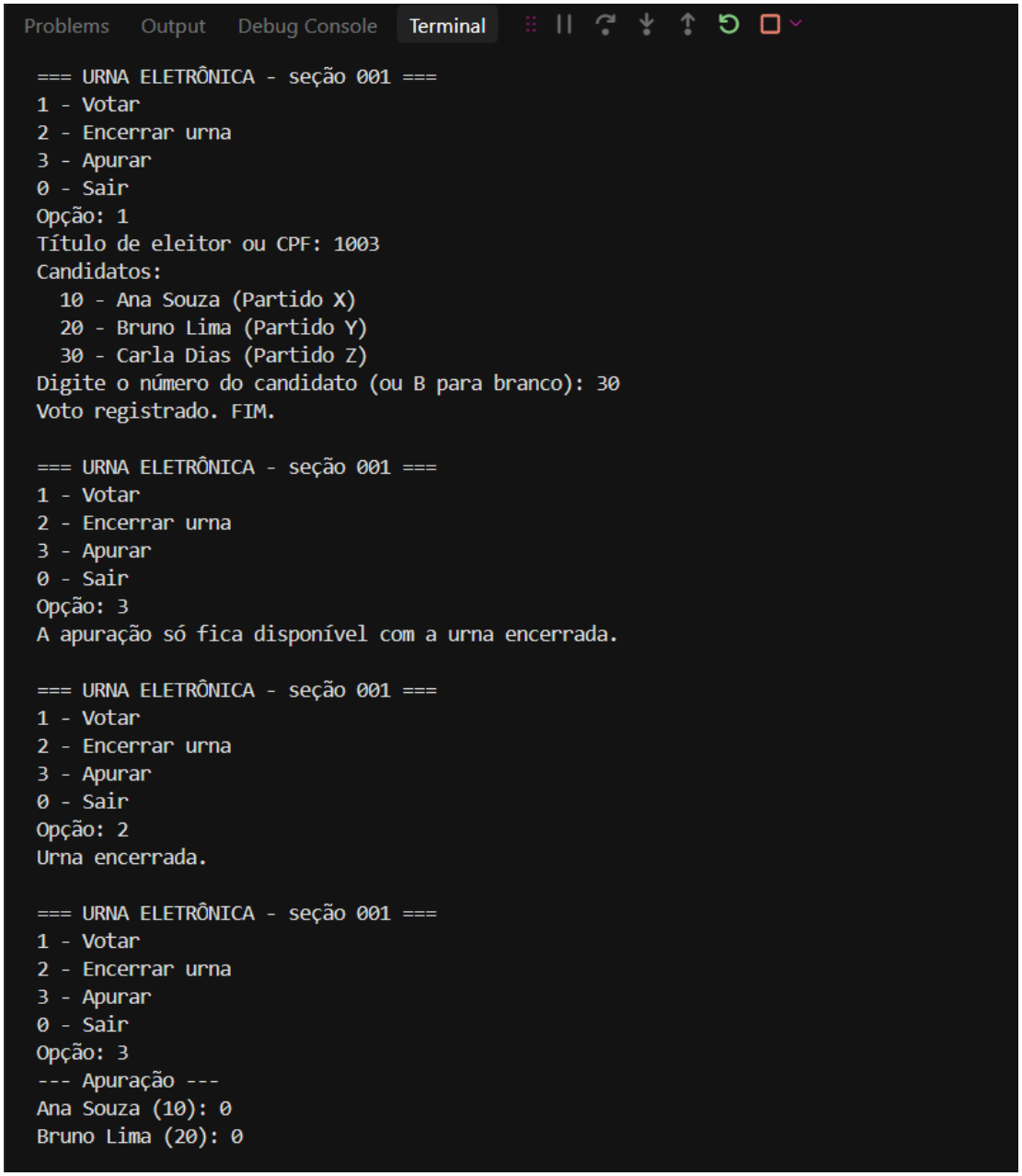


A conferência fecha: votos nos candidatos (3) + brancos (1) + nulos (1) = 5 votos, o mesmo número de eleitores que votaram.

Como rodar: compilar com `javac --release 17 -d out src/modelo/*.java`

`src/app/*.java` e executar `java -cp out app.Demo`. O app.Main é a versão interativa, com menu no console (votar, encerrar e apurar). Nesta entrega compilamos direto com javac; o Maven (pom.xml) entra na Entrega 3, junto com o driver do SQLite. A execução do print foi feita no VS Code com JDK 25, e o projeto segue o Java 17 como versão mínima

Requisitos cobertos nesta entrega, em listas (memória): RF.1 a RF.5 (RF.2 parcial, pois a urna recebe os candidatos pelo construtor) e RN.1 a RN.4. RN.5 e RN.6 ficam parciais: o eleitor só marca jaVotou depois que a urna aceitou o voto, e a checagem de eleitor e candidato é feita em memória. A transação JDBC e a validação pelo banco vêm na Entrega 3.



*Saída do app.Main (versão interativa)

Entrega 3: Persistência e funcionalidades

Objetivo: dados gravados e recuperados, e ao menos metade das funcionalidades do enunciado funcionando.

3.1 Organização em camadas

Liste os pacotes do projeto e a responsabilidade de cada um (por exemplo: modelo, dao ou arquivo, gui ou app).

Pacote	Responsabilidade	Principais classes
modelo		
dao / arquivo		
gui / app		

3.2 Persistência

Descreva como os dados são gravados e lidos (tabelas e DAOs, ou arquivos e formato). Informe como o banco é criado na primeira execução.

Escreva aqui.

3.3 Funcionalidades implementadas

Funcionalidade	Situação (pronta / parcial / pendente)	Responsável

3.4 Decisões e dificuldades

Registre as principais decisões de projeto e os problemas encontrados, com o que foi feito para resolvê-los.

Escreva aqui.

Entrega 4: Versão final

Objetivo: aplicação completa, documentada e pronta para a demonstração.

4.1 Como compilar e executar

Passo a passo para outra pessoa rodar o projeto do zero (versão do Java, comandos, dependências, banco ou arquivos iniciais).

Escreva aqui.

4.2 Telas finais

Capturas das telas (ou do menu) com uma legenda curta explicando cada uma.

Cole aqui as capturas.

4.3 UML final e diferenças

Cole o diagrama final e comente o que mudou em relação à Entrega 1.

Cole aqui o diagrama final e os comentários.

4.4 Tratamento de erros

Liste as situações de erro tratadas e a mensagem exibida ao usuário em cada uma.

Situação de erro	Exceção	Mensagem ao usuário

4.5 Limitações e melhorias futuras

Escreva aqui.

4.6 Contribuição de cada integrante

Descreva o que cada pessoa fez. Este quadro será confrontado com o histórico de commits.

Integrante	Contribuições

Integrante	Contribuições

Referências

Livros, documentação e sites consultados, em formato ABNT ou similar. Indique também qualquer trecho de código adaptado de outra fonte.

Liste aqui as referências.